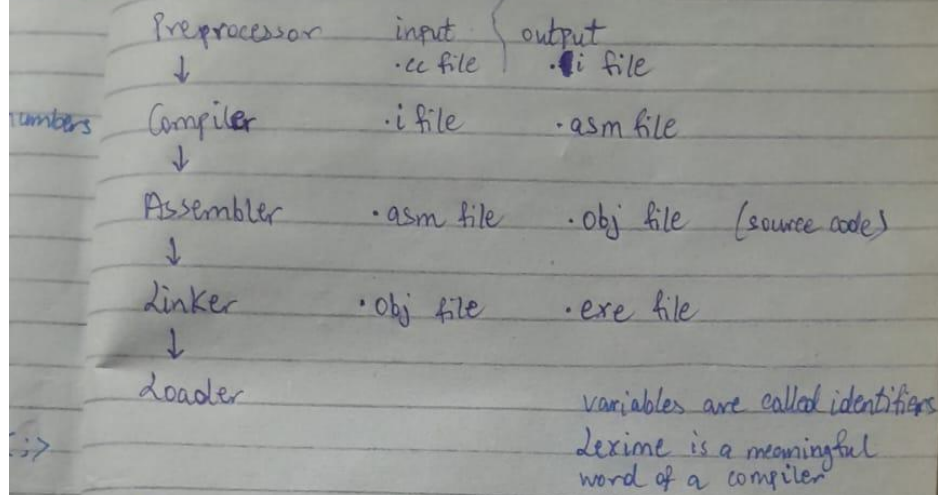


Compiler Construction



All imports for e.g. #include iostream are resolved in preprocessor. Modified source code is .I file

Interpreter executes code line by line but compiler first converts code into executable.

Phases Of Compiler

① Lexical Analysis / Scanner
outputs stream of tokens

Token: < data type, pointer to ST >

ST: Symbol table that saves identifiers

Transition diagrams

RE, RD (regular definitions)

only DFA to be implemented that are called transition diagrams.

code:

position	Lexeme	
1	position	1. float partition;
2	initial	2. float initial;
3	rate	3. float rate;
		4. partition = initial + rate * 60;

Tokens:

method 1: $\langle ID, 1 \rangle$ class 4 position in symbol table

method 2: $\langle = \rangle$ Lexeme only in case of pointers, punctuation, numbers
 $\langle op, = \rangle$

Keyword table can be made separately so that

$\langle class, pointer \rangle$ can be used for keywords.

Literal table to store string or number literals.

Total tokens in line 4 of code:

$\langle ID, 1 \rangle \langle = \rangle \langle ID, 2 \rangle \langle + \rangle \langle ID, 3 \rangle \langle * \rangle \langle 60 \rangle \langle ; \rangle$

Lexical errors such as $[int\ ab\ \$def;]$, in that
'\$' is stored in errors.txt

{ All possible characters } - { set of characters in C language }
= Lexical errors

② Syntax Analysis / Parser

input: stream of tokens

output: Abstract parse tree

we CFC in form of PDA

typical syntax errors such as semicolon missing or
ill structured language

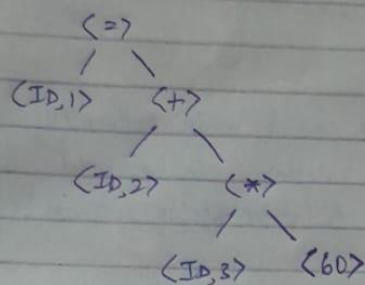
int name; $\rightarrow$ keyword
 $\downarrow \downarrow \downarrow$
keyword ID operator ID

inta;
{
 inta;
}
not an error
scope changed

Syntax Analysis adds ^{data} type column
to symbol table

scope is defined via a new symbol table.
Global variables copied in every symbol table.

parse tree:

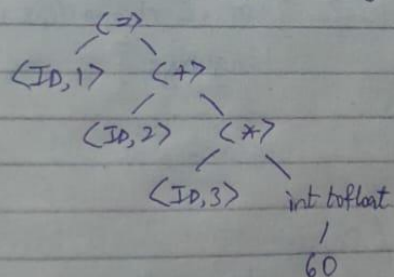


③

Semantic Analysis

output: Annotated Parse tree

Attributed grammar (FL + semantics)
type casting etc. change meaning of operators.



Semantic errors:

Array out of bounds
invalid memory access
redefinition errors

Size of pointer/variable column can be added to
symbol table in this phase or other attributes.

④ Intermediate Code Generation

output: Three Address code

Syntax directed translation

: somewhat like a
CEA

e.g.

t1 = int to float (60)

$t_2 = \langle ID, 3 \rangle * t_1$
 $t_2 = \langle ID, 2 \rangle + t_2$
 $\langle ID, 1 \rangle = t_2$

placeholders such as t_1, t_2 also
get stored in symbol table

tree read bottom to top & converted

First pass of Compiler / Front end Completed
after 4 phases. $\cdot cpp \rightarrow \cdot tac$

Code Optimization

input: $\cdot tac$ output: $\cdot tac$

e.g.

$t_1 = \langle ID, 3 \rangle * \#60.0$
 $\langle ID, 1 \rangle = \langle ID, 2 \rangle + t_1$

reduce no. of
instructions

Three Address code has some rules.

Code Generation

LDR R2, Id3
 MULF R2, R1, #60.0
 LDR R1, Id2
 ADDF R1, R1, R2
 STF Id1, R1

Machine Dependant Code Optimization

Regular Definition

operators: . \ / {} [] () * + ?
 ^ | " " \$ -

? : 0 or 1 occurrence

- : 0 - n range defined e.g. [a-z] or [A-Z]
 only used within [] or [1-5]

| : or operator

+ : 1 or more occurrence

* : 0 or more occurrence

. : any character

^ : new line / start of line e.g. ^int

\$: end of the line e.g. ; \$

AND () : grouping

OR [] : grouping with choice e.g. [ab] means a/b

[^x] : negation of whatever that x is

{ } : no. of occurrences e.g. a{3}, a{3,5}, a{,5}, a{3,}

\ : escape character e.g. \. selects .

lines that start with 'int' and end with ';'
 And: ^int.*;\$

in [^int] neither i, n, t are selected

" " : for entire string search. whatever inside except loses its power & is treated as the exact literal only.

[int]{3,3}

/ : to exactly mention when the pattern starts & ends e.g. /home/ searches for home

Cannot use:

[1] (-)

~~Not strings~~
~~Ex-28~~

Regex to check all variables / identifiers

$[a-zA-Z_][a-zA-Z0-9_]*$

Regular Definitions

small_alpha $\rightarrow [a-z]$

Capital_alpha $\rightarrow [A-Z]$

letter $\rightarrow$ small_alpha | Capital_alpha

Specialcharacter $\rightarrow _$

letter_ $\rightarrow$ letter | Specialcharacter

digit $\rightarrow [0-9]$

we can reuse
regular
definitions

ID $\rightarrow$ letter_ (letter_ | digit)*

Finite Automata

$\textcircled{A} \xrightarrow{\Sigma} \textcircled{B}$ $\Sigma = \{a, b, c\}$

letter = $\{a, b, c, \dots, z, A, B, C, \dots, Z\}$

digit = $\{0, 1, 2, \dots, 9\}$

letter_ = letter $\cup$ $\{-\}$

$\Sigma_{ID} =$ letter_ $\cup$ digit

$|\Sigma_{ID}| = 63$

FA to check variables / identifiers

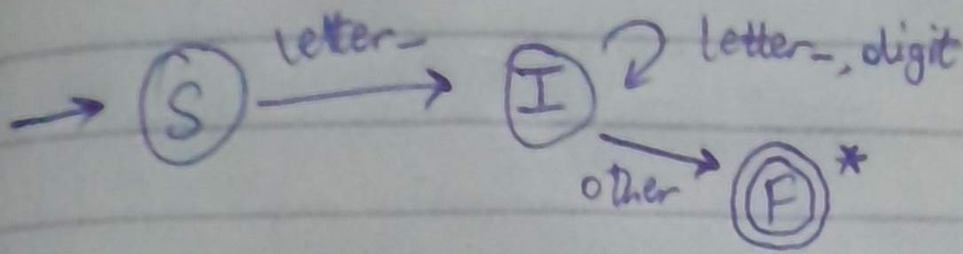
$\rightarrow \textcircled{S} \xrightarrow{\text{letter_}} \textcircled{P} \xrightarrow{\text{letter_ , digit}}$

Not correct for CC

e.g. _num_one
will return 8
tokens but
we need only 1

Transition Diagram

Proper FA for identifiers in CC

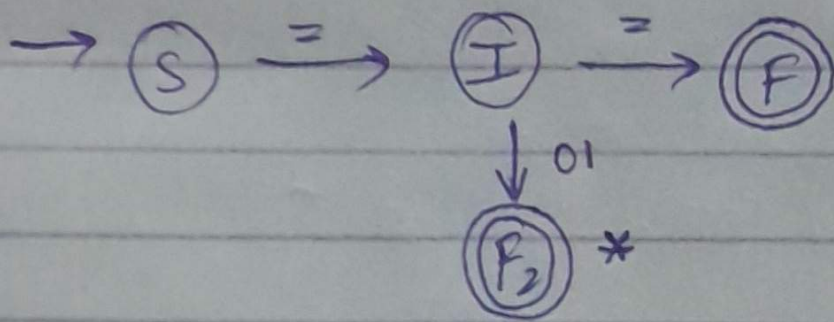


$$\text{Let other} = \Sigma_c - \Sigma_{ID}$$

a * on final state means that pointer is stopped at final state. It "reads" the pointer". If fore.g.

pointer read other & moved to F then pointer will move one step back to allow "other" back into input stream.

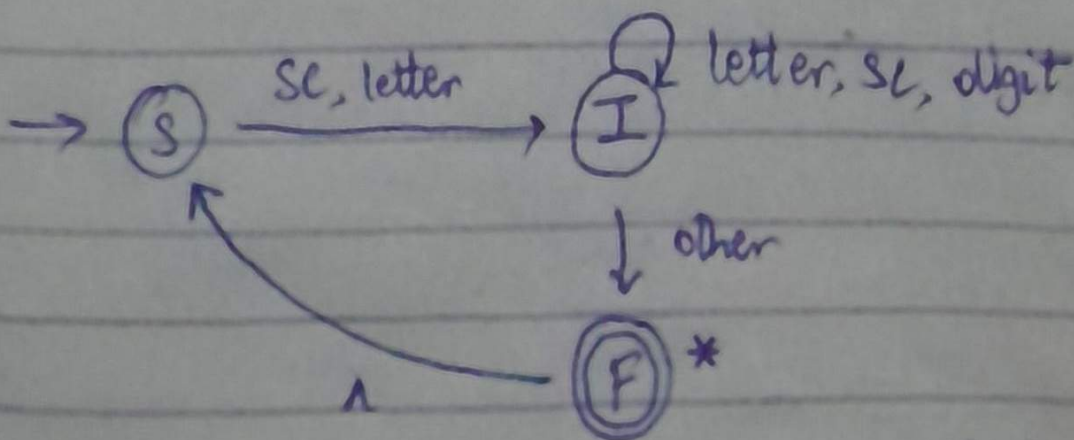
$$L = \{=, ==\}$$



$$01 = \Sigma_c - \{=\}$$

Lexeme Processing Using Static Approach / Ad Hoc lexer

If any change in rules is required then there is change in code as well along with transition diagram. If a dynamic lexer is used, even if transition diagram changes due to redefinition, code does not change.



Λ is important to return to starting state.

Table down approach / Dynamic approach.

Step 1

Mapping

Transition table:

states			Σ				
Q		Σ					
0	S	0 letters		0	1	2	3
1	I	1 SC	Q	0	1	1	
2	F	2 digits		1	1	1	2
		3 other		2			

digits in grid represent state numbers

Advance (pointer)

		Σ			
		0	1	2	3
Q	0	1	1		
	1	1	1	1	0
	2				

digits in this grid represent boolean (yes or no)

Syntax Analyzer / Parser
using CFL

4 tuples Top down parsers - Recursive Descent
221

Bottom up parser - CYK parser (not in CL)

CFL for declaration in code

int x;

float variable;

string name;

$S \rightarrow \text{Keyword ID} ; S \mid \text{Keyword ID} ;$

$\text{Keyword} \rightarrow \text{int} \mid \text{float} \mid \text{string}$